

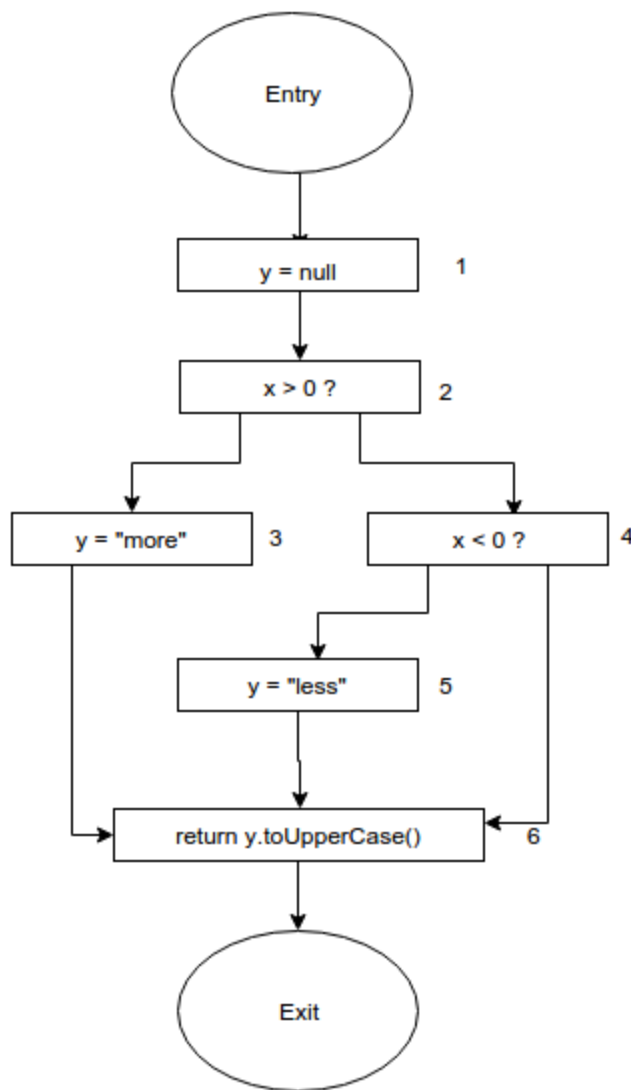
Data Flow Analysis

Data Flow Analysis is a type of static analysis. The goal of *static analysis* is to reason about program behavior at *compile-time*, before ever running the program. The goal of *dynamic analysis*, in contrast, is to reason about program behavior at *run-time*. Data Flow Analysis typically operates over a **Control-Flow Graph (CFG)**, a graphical representation of a program.

Consider, for example, the following Java code:

```
public String badCode(int x) {  
    String y = null;  
    if (x > 0) {  
        y = "more";  
    } else if (x < 0) {  
        y = "less";  
    }  
    return y.toUpperCase();  
}
```

We can represent this by the following CFG:



Using this CFG, we can reason *globally* (from the start to the exit of the CFG) about the behavior of a program by reasoning *locally* (at a single statement or sequence of statements without branches) about facts. For example, we may want to know if there are any possible null-pointer exceptions in our program. We can do this by reasoning locally about the definitions in our CFG. We know program point 1 assigns `null` to a variable, and we also know this value is overwritten at points 3 and 5. Using this information, we can determine whether the definition at point 1 *may reach* program point 6 where it's used. If it can, we have a possible null-pointer exception.

Classifying Data Flow Analyses

Let's revisit this idea formally. There are several ways to reason about a CFG:

- **Forward** or **backward**, or sometimes both
- Whether facts **may** or **must** be true
- Within a procedure (**intraprocedural**) or between procedures (**interprocedural**)

Consider an arbitrary program point p . In a *forward* analysis, we are reasoning about facts *up to* p , considering only the *predecessors* of the node at p . In a *backward* analysis, we are reasoning about facts *from* p *onward*, considering only the successors.

In a *may* analysis, we care about the facts that *may be true* at p . That is, they are true *for some path* up to or from p , depending on the direction of the analysis. In a *must* analysis, we care about the facts that *must be true* at p . That is, they are true *for every path* up to or from p . This manifests when we reach a *join point* in our program. A join point is when multiple paths of a program come together in our CFG. In our example, if we are moving forward, program point 6 is a join point. At these join points, we will either take the *union* (for *may*) or *intersection* (for *must*) of sets of facts.

We also care about the initial sets of facts that are true at the entry or exit (depending on the direction), and initially at every in our out point (also depending on the direction). Finally, at each program point, we *generate* and *kill* facts. We generate facts when we have new information at a program point, and we kill facts when that program point invalidates other information. We call these sets *Gen* and *Kill*.

A Generic Algorithm

From this emerges a generic algorithm. If we are moving *forward*, we have the following:

$$\begin{aligned} \text{In}(s) &= /\backslash_{s' \text{ in pred}(s)} \text{Out}(s') \\ \text{Out}(s) &= \text{Gen}(s) \text{ union } (\text{In}(s) \setminus \text{Kill}(s)) \end{aligned}$$

If we are moving *backward*, we instead have the following:

$$\begin{aligned} \text{Out}(s) &= /\backslash_{s' \text{ in succ}(s)} \text{In}(s') \\ \text{In}(s) &= \text{Gen}(s) \text{ union } (\text{Out}(s) \setminus \text{Kill}(s)) \end{aligned}$$

Here, $/\backslash$ refers to "meet," which is union for *may* analyses and intersection for *must* analyses.

In other words, when we move *forward*, we get our *input* by taking the union or intersection of the *outputs* of all of the *predecessors*, and we get our *output* by reasoning locally about the facts generated and killed at that point. When we move *backward*, we get our *output* by taking the union or intersection of all the *inputs* of all of the *successors*, and we get our *input* by reasoning locally about our facts. The algorithm is executed until all facts converge, that is, until they don't change anymore.

It turns out that data flow facts typically form a *lattice*. You shouldn't have to worry about this for class, but if you're interested in the math behind this, I highly encourage you to read [these slides](#) to find out more, or ask in office hours. The slides also generalize the algorithm more and discuss why we know it must terminate.

Four Classic Analyses

Following are four classic analyses:

- **Available Expressions:** An expression is available at p if it is computed on *every path to* p and has not changed in value since. (**Forward Must**)
- **Very Busy Expressions:** An expression is very busy at p if it is evaluated on *every path from* p before it changes in value. (**Backward Must**)
- **Reaching Definitions:** A definition reaches p if it occurs on *some path to* p with no intervening assignments. (**Forward May**)

- **Live Variables:** A variable is live at p if it is used on *some* path *from* p before it is overwritten. (**Backward May**)

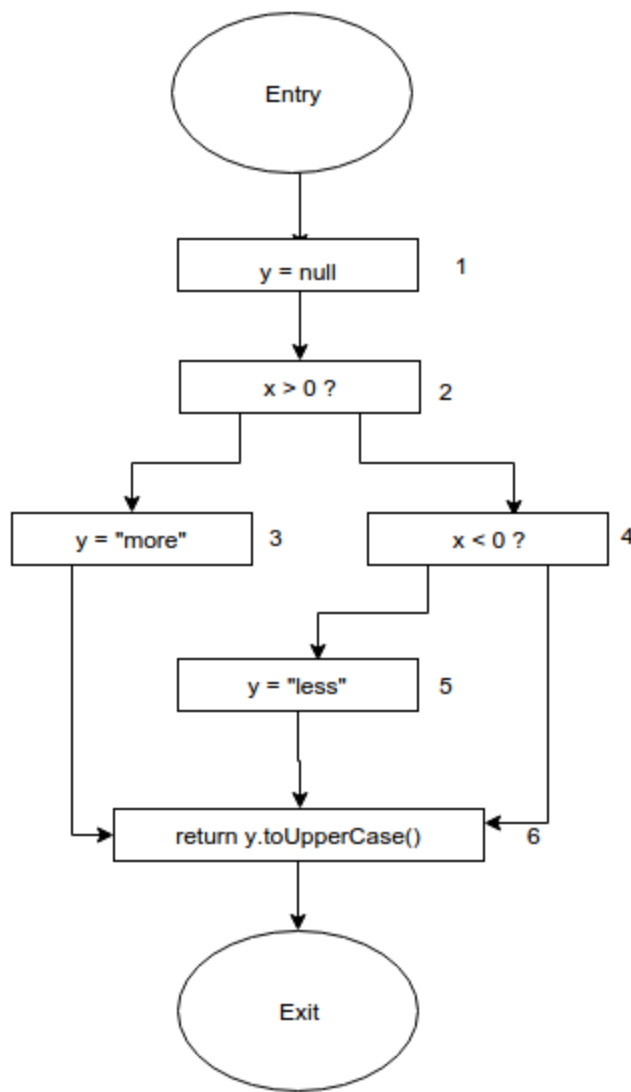
If we look closely at our English definitions, we can also figure out the facts we're reasoning about (the domain of our analysis) and our Gen and Kill sets. In live variables, for example, we are reasoning about variables. A variable is only live if it's *used*, so using a variable in an expression generates information. A variable is only live if it's used *before it is overwritten*, so assigning to the variable kills information.

Each of these analyses has concrete applications in the real world:

- **Available Expressions:** Optimization (don't recompute expressions that are still available)
- **Very Busy Expressions:** Optimization (move expression to a common point)
- **Reaching Definitions:** Bug-finding (uninitialized variables), optimization (constant propagation)
- **Live Variables:** Optimization (don't store variables that aren't live, eliminate assignments where variables are dead)

Reaching Definitions

Let's revisit our CFG from before:



We want to determine if there may be a null-pointer exception at program point 6. In other words, we want to know if the definition at point 1 may reach point 6. We assume for now that `y.toUpperCase()` has no side-effects. We denote the empty set as `{}`; this is analogous to all facts being false as seen in lecture. We consider Reaching Definitions in more detail:

- **Facts:** Definitions, which we write as program points where assignments occur
- **Direction:** Forward
- **May/Must:** May (union at join points)
- **Entry facts:** `{}`
- **Initial facts** (referred to as initialization in lecture): `{}` at all input points
- **Gen:** `{p}` if there is a definition to a variable at p , otherwise `{}`
- **Kill:** If there is an assignment to a variable at p , all other points p' that assign to the same variable, otherwise `{}`

Now, let's run through our algorithm:

Program Point	Gen	Kill	In	Out

1	{1}	{3,5}	{}	{1}
2	{}	{}	{1}	{1}
3	{3}	{1,5}	{1}	{3}
4	{}	{}	{1}	{1}
5	{5}	{1,3}	{1}	{5}
6	{}	{}	{1,3,5}	{1,3,5}

It turns out that the definition $y = \text{null}$ at program point 1 can reach program point 6, so y may still be null at program point 6. This means we have a possible null-pointer exception!

To verify the stronger assertion that `y.toUpperCase()` cannot be a null dereference (which, in this case, we know is not true), we need more than just a Reaching Definitions analysis. We need to consider assignments of the form $p = r$, $p = r.f$, and $r.f = p$. This is where pointer analysis comes in handy. See the [Lecture Slides](#) for more details.